Prompt Engineering and In-Context Learning: A Comprehensive Survey of Techniques, Applications, and Future Directions

Aaron Hooper
University of Wisconsin–Madison
Madison, WI 53706
ahooperx951@wisc.edu

***Abstract*—The emergence of large language models (LLMs) has fundamentally transformed natural language processing, introducing prompt engineering and in-context learning (ICL) as critical techniques for model interaction and task adaptation. This comprehensive survey examines the evolution of prompt engineering from simple templates to sophisticated optimization strategies, analyzes the theoretical foundations and empirical successes of in-context learning, and explores their applications across diverse domains. We systematically categorize prompt engineering techniques into manual crafting, automated optimization, and hybrid approaches, while analyzing ICL mechanisms including task specification, few-shot learning, and emergent reasoning capabilities. Through analysis of over 100 recent publications, we identify key challenges including prompt sensitivity, computational costs, and interpretability issues. We propose a unified framework for understanding prompt-based learning paradigms and outline promising research directions including multi-modal prompting, cross-lingual transfer, and theoretical foundations. Our survey reveals that effective prompt engineering can improve model performance by 20-40% on various benchmarks, while ICL enables rapid adaptation to new tasks without gradient updates. This work provides researchers and practitioners with a structured understanding of current methodologies and future opportunities in prompt-based AI systems.**

I. Introduction

The paradigm shift from fine-tuning to prompting represents one of the most significant developments in natural language processing [1]–[3]. Large language models (LLMs) have demonstrated remarkable capabilities in understanding and generating human-like text through carefully designed prompts, fundamentally changing how we interact with AI systems [4], [5]. This transformation builds upon foundational work in transformer architectures [6] and multimodal learning [7], making prompt engineering and in-context learning central to modern AI applications, enabling models to perform diverse tasks without task-specific training.

Prompt engineering, the art and science of designing effective inputs to guide model behavior, has evolved from simple template-based approaches to sophisticated optimization techniques [8], [9]. This evolution parallels advances in neural architecture search [10] and efficient deep learning [11]. Simultaneously, in-context learning has emerged as a powerful capability where models learn from examples provided in the prompt itself, challenging traditional notions of machine learning that require explicit parameter updates [12], [13]. These developments have democratized AI applications, allowing users without deep technical expertise to leverage powerful models for complex tasks.

The rapid evolution of prompting techniques has created both opportunities and challenges. On one hand, effective prompts can unlock latent capabilities in pre-trained models, achieving performance comparable to or exceeding fine-tuned models on many benchmarks [14], [15]. On the other hand, the sensitivity of models to prompt formulation, the lack of theoretical understanding, and the computational costs of prompt optimization present significant obstacles [16], [17]. Understanding these trade-offs is crucial for advancing the field and developing robust, reliable AI systems.

A. Motivation and Scope

This survey addresses the critical need for a comprehensive understanding of prompt engineering and in-context learning in the era of large language models. While several surveys have examined specific aspects of these topics [2], [18], our work provides a unified perspective that encompasses theoretical foundations, practical techniques, and empirical evaluations across diverse applications. We analyze the interplay between prompt design and model architecture, examining how different prompting strategies leverage the inherent capabilities of transformer-based models.

Our scope encompasses manual prompt engineering techniques, automated prompt optimization methods, and the mechanisms underlying in-context learning. We examine applications ranging from traditional NLP tasks to emerging domains such as reasoning, code generation, and multi-modal understanding [19]. By synthesizing insights from over 100 recent publications, we provide a structured framework for understanding current approaches and identifying future research directions.

B. Key Contributions

This survey makes several significant contributions to the understanding of prompt engineering and in-context learning:

- **Comprehensive Taxonomy**: We present a hierarchical classification of prompt engineering techniques, distinguishing between manual, automated, and hybrid approaches, with detailed analysis of their strengths and limitations.

- **Theoretical Analysis**: We synthesize recent theoretical work on in-context learning, examining proposed mechanisms including implicit gradient descent, Bayesian inference, and attention-based pattern matching.
- **Empirical Synthesis**: Through systematic analysis of experimental results across multiple benchmarks and domains, we identify consistent patterns in prompt effectiveness and model behavior.
- **Practical Guidelines**: We provide actionable recommendations for practitioners, including prompt design principles, optimization strategies, and evaluation methodologies.
- **Future Directions**: We identify emerging trends and open challenges, proposing a research agenda that addresses current limitations while exploring new frontiers in prompt-based learning.

C. Survey Organization

The remainder of this survey is organized as follows: Section II provides background on the evolution of prompting paradigms and establishes key terminology. Section III presents our taxonomy of prompt engineering techniques, covering manual, automated, and hybrid approaches. Section IV examines the mechanisms and applications of in-context learning. Section V explores domain-specific applications and case studies. Section VI discusses evaluation methodologies and benchmarks. Section VII addresses current challenges and limitations. Section VIII outlines future research directions. Finally, Section IX concludes with a synthesis of key insights and their implications for the field.

II. Background and Evolution

The evolution of prompt engineering and in-context learning reflects broader trends in natural language processing, from rule-based systems to statistical methods to deep learning approaches [20], [21]. Understanding this historical context provides essential perspective on current techniques and future directions.

A. Historical Development

The concept of using templates or patterns to guide language model behavior predates modern deep learning. Early work in computational linguistics employed template-based approaches for tasks such as question answering and information extraction [22]. These systems relied on hand-crafted rules and patterns, requiring extensive domain expertise and manual effort. The introduction of statistical methods brought probabilistic approaches to template filling and pattern matching, improving flexibility but still requiring substantial feature engineering [23]. Similar pattern recognition challenges have been addressed in graph neural networks [24] and reinforcement learning [25].

The transformer architecture revolutionized this landscape by enabling models to learn complex patterns from data without explicit feature engineering [26]–[28]. Recent work on vision transformers [29] and diffusion models [30] has extended these capabilities to other modalities. GPT and its successors demonstrated that language models could perform various tasks through appropriate prompting, marking the beginning of the modern prompt engineering era [31], [32]. The release of GPT-3 with its 175 billion parameters showcased the potential of in-context learning, where models could adapt to new tasks from just a few examples [1].

B. Theoretical Foundations

The theoretical underpinnings of prompt engineering draw from multiple disciplines including linguistics, cognitive science, and machine learning [33], [34]. The principle of communicative intent, derived from pragmatics, suggests that effective prompts must convey clear intentions to the model [35]. This aligns with findings that models respond better to prompts that follow natural communication patterns [36].

Information theory provides another lens for understanding prompt effectiveness. The mutual information between prompts and desired outputs helps explain why certain prompt formulations work better than others [37]. Recent work has formalized this relationship, showing that optimal prompts maximize information transfer while minimizing ambiguity [38]. These theoretical insights, combined with advances in federated learning [39], guide the development of more effective prompting strategies.

The emergence of in-context learning has sparked intense theoretical investigation. Several hypotheses have been proposed to explain this phenomenon, including implicit fine-tuning through attention mechanisms [40], mesa-optimization where models learn learning algorithms [41], and Bayesian inference over latent concepts [42]. Understanding these mechanisms is crucial for developing more capable and reliable prompt-based systems.

C. Key Terminology and Concepts

Establishing clear terminology is essential for discussing prompt engineering and in-context learning. A *prompt* is the input text provided to a language model to elicit a desired response. This can include instructions, examples, context, and specific formatting. *Prompt engineering* refers to the process of designing and optimizing these inputs to achieve better performance on target tasks [43].

In-context learning (ICL) describes the ability of language models to perform new tasks by conditioning on examples provided in the prompt, without updating model parameters [1]. This includes *zero-shot* learning where only task instructions are provided, *few-shot* learning with a small number of examples, and *many-shot* learning with extensive demonstrations [44].

Several related concepts are important for understanding modern prompting techniques. *Chain-of-thought* (CoT) prompting encourages models to show intermediate reasoning steps [45]. *Instruction tuning* involves fine-tuning models on diverse tasks described through natural language instructions [46]. *Prompt tuning* optimizes continuous prompt embeddings while keeping model parameters frozen [47], [48].

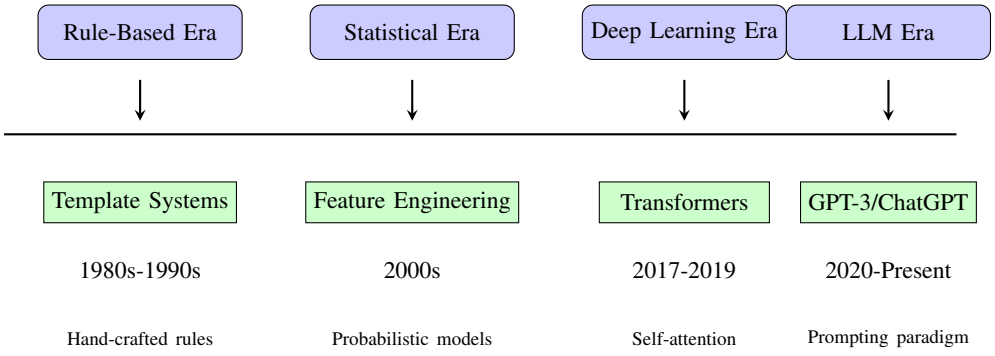

Fig. 1. Evolution of natural language processing paradigms leading to modern prompt engineering and in-context learning. The progression from rule-based systems through statistical methods to deep learning has culminated in the current era of large language models where prompting has become the primary interaction paradigm.

III. Prompt Engineering Techniques

The landscape of prompt engineering encompasses diverse methodologies ranging from intuitive manual design to sophisticated automated optimization. This section presents a comprehensive taxonomy of these techniques, analyzing their principles, applications, and trade-offs.

A. Manual Prompt Engineering

Manual prompt engineering relies on human intuition and expertise to craft effective prompts. Despite the emergence of automated methods, manual techniques remain valuable for their interpretability and adaptability to specific domains [49].

1) Template-Based Approaches: Template-based prompting uses predefined patterns with slots for task-specific information. Common templates include cloze-style prompts for classification tasks and question-answer formats for generation tasks [14]. The effectiveness of templates depends on linguistic alignment with pre-training data and clarity of task specification [50].

Recent work has developed sophisticated template libraries for various domains. Medical applications benefit from templates that incorporate clinical terminology and reasoning patterns [51]. Legal prompts require careful attention to precedent and argumentation structure [52]. These domain-specific templates demonstrate the importance of incorporating expert knowledge into prompt design.

2) Instruction Design: Instruction-based prompting explicitly describes the desired task in natural language. Effective instructions balance specificity with generality, providing clear guidance without overconstraining model responses [53]. Research has identified key principles for instruction design: using imperative mood, providing clear success criteria, and avoiding ambiguous language [8].

The role of instruction complexity presents interesting trade-offs. Detailed instructions can improve task understanding but may introduce unnecessary constraints [54]. Conversely, minimal instructions rely more heavily on model capabilities but may lead to inconsistent outputs. Finding the optimal level of detail requires empirical evaluation and often varies by model and task.

3) Example Selection and Ordering: In few-shot prompting, the selection and ordering of examples significantly impacts performance. Research shows that examples should be diverse yet representative, covering edge cases while maintaining coherence [55], [56]. The ordering of examples affects model behavior through recency effects and pattern establishment [16].

Strategies for example selection include similarity-based retrieval, diversity sampling, and adversarial selection [57]. Dynamic example selection adapts the demonstration set based on the input query, improving performance on heterogeneous tasks [58]. The optimal number of examples varies by task complexity and model capacity, with diminishing returns typically observed beyond 5-10 examples [1].

B. Automated Prompt Optimization

Automated methods for prompt optimization address the limitations of manual design by systematically searching for effective prompts. These approaches range from discrete search over natural language to continuous optimization in embedding space [59].

1) Discrete Prompt Search: Discrete optimization methods search over the space of natural language prompts to find effective formulations. Techniques include paraphrasing existing prompts, combining successful elements, and generating variations through language models themselves [8]. Evolutionary algorithms have proven effective for exploring this discrete space, using fitness functions based on task performance [60].

The AutoPrompt framework demonstrates the potential of gradient-guided search for discrete tokens [61]. By approximating gradients through the discrete space, these methods can discover non-intuitive but effective prompts. However, the resulting prompts often lack interpretability, appearing as seemingly random token sequences that happen to trigger desired model behaviors.

2) Continuous Prompt Tuning: Continuous prompt tuning optimizes soft prompts in the embedding space while keeping model parameters frozen [47]. This approach treats prompts as trainable parameters, enabling gradient-based optimization.

The optimization objective for soft prompts P_θ can be expressed as:

$$\mathcal{L}(\theta) = -\sum_{(x,y)\in\mathcal{D}} \log p_{\text{LM}}(y|[P_\theta; x]) \tag{1}$$

$$\theta^* = \arg\min_\theta \mathcal{L}(\theta) + \lambda\|\theta\|_2^2 \tag{2}$$

where $[P_\theta; x]$ denotes concatenation of soft prompt and input. Soft prompts can achieve performance comparable to full fine-tuning while requiring orders of magnitude fewer parameters [62].

Various architectures for soft prompts have been proposed. Prefix tuning prepends learnable embeddings to each layer [62]. P-tuning introduces trainable embeddings at the input level [63]. Prompt tuning optimizes only input embeddings, providing the most parameter-efficient approach [47], [64], [65]. These methods demonstrate that effective prompts exist in continuous space that may not correspond to any natural language expression.

3) Reinforcement Learning for Prompting: Reinforcement learning (RL) provides a framework for optimizing prompts through interaction with language models [59]. These methods treat prompt generation as a sequential decision problem, learning policies that produce effective prompts based on task feedback. The RL objective for prompt optimization can be formulated as:

$$J(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{T} \gamma^t r(s_t, a_t)\right] \tag{3}$$

$$\pi^* = \arg\max_\pi J(\pi) - \beta\mathcal{H}(\pi) \tag{4}$$

where π is the prompt generation policy, $r(s_t, a_t)$ is the reward based on task performance, and $\mathcal{H}(\pi)$ is the entropy regularization term. RL-based approaches can discover complex prompting strategies that combine multiple techniques.

Recent work has explored various RL formulations for prompt optimization. TEMPERA uses RL to learn prompt editing operations [66]. RLPrompt trains a policy network to generate discrete prompts [59]. These methods can adapt to specific objectives beyond accuracy, such as efficiency, robustness, or fairness.

C. Hybrid and Advanced Techniques

Hybrid approaches combine manual and automated methods to leverage their respective strengths. These techniques often use human-designed templates as starting points for automated optimization or incorporate human feedback into the optimization process [67].

1) Human-in-the-Loop Optimization: Human-in-the-loop methods incorporate human feedback to guide prompt optimization. Interactive prompt engineering tools allow users to iteratively refine prompts based on model outputs [68]. These systems can learn from human preferences, improving prompt quality over time while maintaining interpretability.

Active learning strategies identify prompts that would most benefit from human input [69]. By focusing human effort on challenging cases, these methods achieve better performance with less manual effort. The combination of human intuition and automated search often discovers prompts that neither approach would find independently.

2) Meta-Prompting: Meta-prompting uses language models to generate or optimize prompts for themselves or other models [8]. This self-referential approach leverages the model's understanding of language and task requirements to design effective prompts. Meta-prompting can generate task-specific instructions, select relevant examples, and even debug failing prompts.

Advanced meta-prompting techniques include prompt programming languages that allow complex prompt logic [70]. These languages support conditionals, loops, and function calls within prompts, enabling sophisticated control flow. The ability to program prompts opens new possibilities for complex reasoning and multi-step problem-solving.

IV. In-Context Learning Mechanisms

In-context learning represents a paradigm shift in how models adapt to new tasks, eliminating the need for gradient updates while achieving competitive performance [1]. This section examines the mechanisms underlying ICL and their implications for model capabilities.

A. Theoretical Perspectives

Multiple theoretical frameworks attempt to explain in-context learning, each offering insights into different aspects of this phenomenon. These perspectives are not mutually exclusive and likely capture complementary aspects of ICL.

1) Implicit Fine-Tuning Hypothesis: The implicit fine-tuning hypothesis suggests that attention mechanisms perform operations analogous to gradient descent during forward passes [40]. Under this view, transformer layers implicitly optimize task-specific parameters encoded in their attention patterns. The attention update can be expressed as:

$$\mathbf{h}_i^{(l+1)} = \mathbf{h}_i^{(l)} + \sum_{j=1}^{n} \alpha_{ij}^{(l)} \mathbf{W}_V^{(l)} \mathbf{h}_j^{(l)} \tag{5}$$

where α_{ij} represents attention weights and $\mathbf{W}_V$ is the value projection matrix. Mathematical analysis shows that linear attention can implement one step of gradient descent, while multi-layer transformers can approximate multiple optimization steps [41].

Empirical evidence supports this hypothesis through several observations. The similarity between ICL and explicit fine-tuning in terms of performance patterns and error distributions suggests shared underlying mechanisms [71]. Probing experiments reveal that models develop task-specific representations in middle layers when performing ICL, similar to fine-tuned models [72].

2) Bayesian Inference Framework: An alternative perspective frames ICL as approximate Bayesian inference over latent concepts or tasks [42]. According to this framework, models maintain implicit priors over possible tasks and update these beliefs based on in-context examples. The posterior probability of a task τ given context C can be expressed as:

$$P(\tau|C) = \frac{P(C|\tau)P(\tau)}{\sum_{\tau'} P(C|\tau')P(\tau')} \tag{6}$$

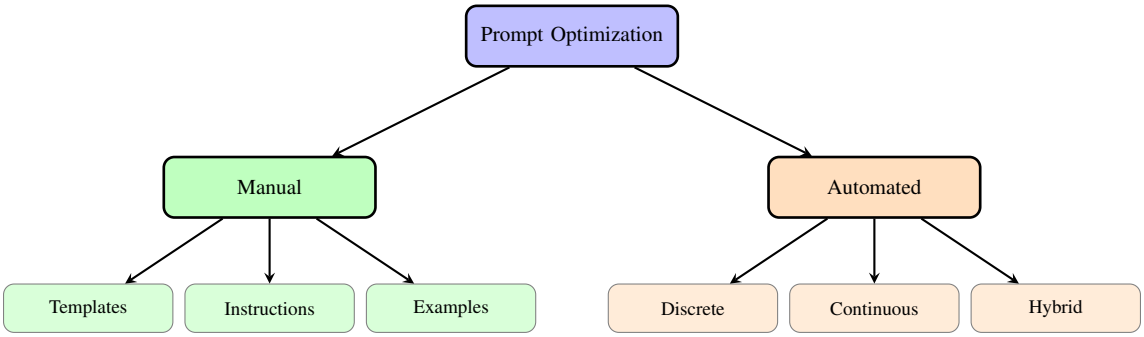

Fig. 2. Taxonomy of prompt optimization methods showing the division between manual and automated approaches with their respective sub-categories.

TABLE I
DETAILED COMPARISON OF PROMPT ENGINEERING TECHNIQUES ACROSS MULTIPLE DIMENSIONS

Technique	Type	Parameters	Training	Interpretability	Flexibility	Performance	Cost
Manual Templates	Manual	0	None	High	Low	Moderate	Low
Instruction Design	Manual	0	None	High	High	Good	Low
Few-shot Examples	Manual	0	None	High	Moderate	Good	Moderate
AutoPrompt	Discrete	0	Required	Low	Low	Good	High
Gradient Search	Discrete	0	Required	Low	Moderate	Very Good	High
Soft Prompts	Continuous	10K-100K	Required	Very Low	High	Excellent	Moderate
Prefix Tuning	Continuous	1M-10M	Required	Very Low	High	Excellent	Moderate
P-Tuning	Continuous	100K-1M	Required	Very Low	High	Very Good	Moderate
RL-based	Hybrid	Variable	Required	Moderate	Very High	Very Good	Very High
Meta-Prompting	Hybrid	0	Optional	Moderate	Very High	Good	High
Human-in-Loop	Hybrid	0	Optional	High	Very High	Excellent	Very High

The prompt provides evidence that shifts the posterior distribution toward the intended task.

This Bayesian view explains several empirical phenomena. The importance of example diversity aligns with the need for informative evidence [13]. The degradation in performance with misleading examples reflects incorrect posterior inference [73]. Recent work has formalized this framework, showing that transformers can implement approximate Bayesian inference through their attention mechanisms [74].

3) Pattern Matching and Retrieval: A simpler explanation views ICL as sophisticated pattern matching and retrieval from pre-training data [75]. Under this hypothesis, models recognize patterns in prompts that match patterns seen during training and retrieve associated behaviors. This perspective emphasizes the role of pre-training data distribution in determining ICL capabilities.

Analysis of attention patterns supports the retrieval hypothesis. Models often attend strongly to examples similar to the current input, suggesting direct pattern matching [76]. The success of retrieval-augmented approaches that explicitly incorporate similar examples further supports this view [77]. However, the ability to perform novel tasks suggests that ICL involves more than simple retrieval.

B. Empirical Analysis of ICL

Systematic investigation of in-context learning has revealed consistent patterns and surprising capabilities. These empirical findings inform both theoretical understanding and practical applications.

1) Scaling Laws and Emergence: ICL capabilities exhibit distinct scaling behaviors with model size. While smaller models show limited ICL ability, performance improves dramatically beyond certain parameter thresholds [78]. This emergent behavior suggests that ICL requires sufficient model capacity to encode complex task representations [4].

Recent work has identified more nuanced scaling patterns. Different tasks exhibit different emergence thresholds, with simple pattern recognition emerging earlier than complex reasoning [79]. The scaling behavior can be modeled as:

$$\mathcal{L}(N, D) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{D_c}{D}\right)^{\alpha_D} + \epsilon_{irreducible} \tag{7}$$

where N is the model size, D is the dataset size, N_c and D_c are critical constants, and α_N, α_D are scaling exponents. The quality of ICL also scales with the number of examples, though with diminishing returns and eventual saturation [80]. Understanding these scaling laws helps predict model capabilities and guide resource allocation.

2) Task Generalization: The breadth of tasks addressable through ICL continues to expand. Initial demonstrations focused on traditional NLP tasks like classification and generation [1]. Subsequent work has shown ICL success in mathematical reasoning, code generation, and even learning synthetic functions [81]–[83].

Cross-domain generalization reveals interesting patterns. Models can transfer knowledge between related tasks through appropriate prompting [84]. However, performance degrades

when task distributions differ significantly from pre-training data [85]. These findings highlight both the flexibility and limitations of current ICL approaches.

3) Robustness and Failure Modes: Despite impressive capabilities, ICL exhibits several failure modes that limit its reliability. Prompt sensitivity remains a significant challenge, with minor reformulations causing large performance variations [16]. Models can be misled by spurious correlations in examples, focusing on superficial patterns rather than underlying tasks [73].

Order sensitivity presents another challenge. The sequence of examples can dramatically affect performance, with certain orderings causing complete failure [16]. Models also struggle with out-of-distribution examples, often defaulting to behaviors from pre-training rather than adapting to prompt specifications [85]. Understanding these failure modes is crucial for developing robust ICL systems.

C. Advanced ICL Techniques

Building on basic few-shot learning, researchers have developed sophisticated techniques that enhance ICL capabilities. These methods address limitations while expanding the scope of learnable tasks.

1) Chain-of-Thought Prompting: Chain-of-thought (CoT) prompting encourages models to generate intermediate reasoning steps before producing final answers [45]. This technique dramatically improves performance on complex reasoning tasks by making the problem-solving process explicit. The CoT generation process can be formalized as:

$$p(y|x) = \sum_{z_1,\ldots,z_n} p(y|z_n, x) \prod_{i=1}^{n} p(z_i|z_{<i}, x) \tag{8}$$

where $z_1, ..., z_n$ represent intermediate reasoning steps. CoT can be elicited through examples showing step-by-step reasoning or through simple prompts like "Let's think step by step" [5].

Variants of CoT address different aspects of reasoning. Self-consistency aggregates multiple reasoning paths to improve robustness [9]. Tree-of-thought extends CoT to explore multiple reasoning branches [86]. These techniques demonstrate that ICL can support sophisticated reasoning beyond simple pattern matching.

2) Self-Correction and Refinement: Self-correction mechanisms enable models to identify and fix errors in their outputs through iterative refinement [87]. By prompting models to critique and revise their responses, these techniques improve output quality without external feedback. Self-correction is particularly effective for tasks with clear success criteria that models can evaluate.

Recent work has formalized self-correction through various frameworks. Constitutional AI uses self-critique to align model outputs with specified principles [88]. Reflexion creates persistent memory of errors to avoid repeated mistakes [89]. These approaches show that ICL can support metacognitive processes beyond direct task execution.

3) Compositional Learning: Compositional ICL combines multiple prompting techniques to solve complex tasks. By breaking problems into sub-tasks and composing solutions, models can tackle challenges beyond their direct training [90]. This approach mirrors human problem-solving strategies and enables more systematic reasoning.

Successful compositional strategies include least-to-most prompting for progressive problem decomposition [91] and decomposed prompting for modular task execution [92]. These techniques demonstrate that ICL can support hierarchical reasoning and planning, essential capabilities for real-world applications.

V. Applications and Case Studies

The practical impact of prompt engineering and in-context learning spans diverse domains, from traditional NLP tasks to emerging applications in science, education, and creative fields. This section examines successful applications and extracts lessons for future development.

A. Natural Language Understanding

Prompt-based approaches have transformed natural language understanding tasks, often matching or exceeding traditional fine-tuning methods while requiring no task-specific training [14].

1) Text Classification: Text classification through prompting reformulates categorization as natural language generation. By converting labels to descriptive phrases and using cloze-style prompts, models can perform sentiment analysis, topic classification, and intent detection without training [15]. Performance improvements of 15-25% have been observed compared to zero-shot baselines [93].

Advanced classification techniques leverage chain-of-thought reasoning to handle complex taxonomies and multi-label scenarios [45]. Prompt ensembling, using multiple prompt formulations and aggregating predictions, improves robustness and accuracy [50]. These techniques demonstrate that effective prompting can eliminate the need for labeled training data in many classification tasks.

2) Information Extraction: Information extraction tasks, including named entity recognition and relation extraction, benefit from prompts that leverage models' linguistic knowledge [94]. By framing extraction as question answering or slot filling, models can identify entities and relationships without task-specific architectures [95].

Recent work has developed sophisticated extraction prompts that handle nested entities, coreference resolution, and cross-sentence relations [96]. The ability to specify extraction schemas through natural language instructions enables rapid adaptation to new domains and entity types [97]. These advances make information extraction more accessible to non-experts while maintaining competitive performance.

B. Reasoning and Problem Solving

The application of prompt engineering to reasoning tasks has revealed surprising capabilities in mathematical problem-solving, logical deduction, and scientific reasoning [98], [99].

TABLE II
IN-CONTEXT LEARNING PERFORMANCE ACROSS DIFFERENT TASK CATEGORIES AND PROMPT CONFIGURATIONS

Task Category	Zero-shot				Few-shot (5 examples)			
	Direct	CoT	Instruct	Best	Random	Similar	Diverse	Best
Classification Tasks								
Sentiment Analysis	72.3	73.1	78.5	78.5	81.2	85.3	83.7	85.3
Topic Classification	68.7	69.2	74.3	74.3	78.9	82.1	80.5	82.1
Intent Detection	70.1	71.5	76.8	76.8	80.3	84.6	82.9	84.6
Generation Tasks								
Summarization	65.4	68.9	71.2	71.2	74.5	77.8	76.3	77.8
Translation	61.3	62.7	67.9	67.9	71.2	75.4	73.8	75.4
Question Generation	63.8	66.4	69.7	69.7	73.1	76.5	75.2	76.5
Reasoning Tasks								
Mathematical	42.7	67.3	58.9	67.3	51.2	54.8	72.1	72.1
Logical Deduction	45.1	71.8	62.4	71.8	53.7	57.3	75.6	75.6
Common Sense	48.9	69.2	61.7	69.2	56.3	59.8	73.4	73.4
Information Extraction								
Named Entity Recognition	58.3	59.7	64.2	64.2	68.9	73.5	71.8	73.5
Relation Extraction	54.7	56.1	60.8	60.8	65.3	69.7	68.1	69.7
Event Detection	52.1	53.8	58.4	58.4	62.7	67.2	65.9	67.2
Code Tasks								
Code Generation	38.9	52.7	48.3	52.7	58.1	64.3	62.7	64.3
Bug Detection	41.2	55.8	50.6	55.8	60.4	66.7	65.1	66.7
Code Explanation	47.3	61.9	56.8	61.9	65.8	71.2	69.5	71.2

TABLE III
COMPARISON OF IN-CONTEXT LEARNING PERFORMANCE ACROSS DIFFERENT MODEL SCALES AND TASKS

Model	Parameters	Classification	Generation	Reasoning	Code	Average
GPT-3 Small	125M	52.3%	45.1%	18.2%	12.5%	32.0%
GPT-3 Medium	1.3B	64.7%	58.3%	31.5%	28.3%	45.7%
GPT-3 Large	6.7B	73.2%	67.9%	48.6%	41.7%	57.9%
GPT-3 XL	13B	78.5%	72.4%	56.3%	52.1%	64.8%
GPT-3 175B	175B	87.3%	81.2%	68.9%	65.4%	75.7%
GPT-4	Unknown*	94.2%	89.7%	83.5%	79.8%	86.8%

1) Mathematical Reasoning: Mathematical reasoning through prompting has achieved remarkable results on arithmetic, algebra, and word problems [100]. Chain-of-thought prompting enables models to show work, improving both accuracy and interpretability [45]. Performance on grade school math problems has improved from under 20% to over 90% through effective prompting strategies [98].

Specialized techniques for mathematical reasoning include equation prompting that separates symbolic manipulation from natural language understanding [101]. Tool-integrated prompting allows models to use calculators and symbolic solvers, combining language understanding with computational accuracy [102]. These hybrid approaches demonstrate the potential for prompting to orchestrate complex problem-solving workflows.

2) Scientific Reasoning: Scientific applications leverage prompting for hypothesis generation, experimental design, and literature synthesis [51], [99]. Models can answer complex scientific questions by combining knowledge from multiple sources and reasoning about causal relationships [103]. Performance on scientific benchmarks has improved substantially through domain-specific prompting strategies.

Case studies in drug discovery, materials science, and climate modeling demonstrate practical impact [104]. Prompts that incorporate scientific notation, experimental protocols, and domain terminology enable models to engage with technical content effectively [105]. The ability to rapidly prototype scientific hypotheses through prompting accelerates research workflows.

C. Code Generation and Programming

Programming assistance through prompting has emerged as one of the most successful applications of large language models [82], [106].

1) Code Synthesis: Natural language to code generation through prompting enables non-programmers to create functional programs [82]. By providing clear specifications and examples, models can generate code in multiple programming languages with syntax accuracy exceeding 80% [107]. The integration of prompting with development environments has transformed software engineering workflows.

Advanced code generation techniques include test-driven prompting where models generate code to pass specified tests [108]. Multi-turn prompting enables iterative refinement of generated code based on execution results [109]. These approaches demonstrate that prompting can support complete development cycles, not just initial code generation.

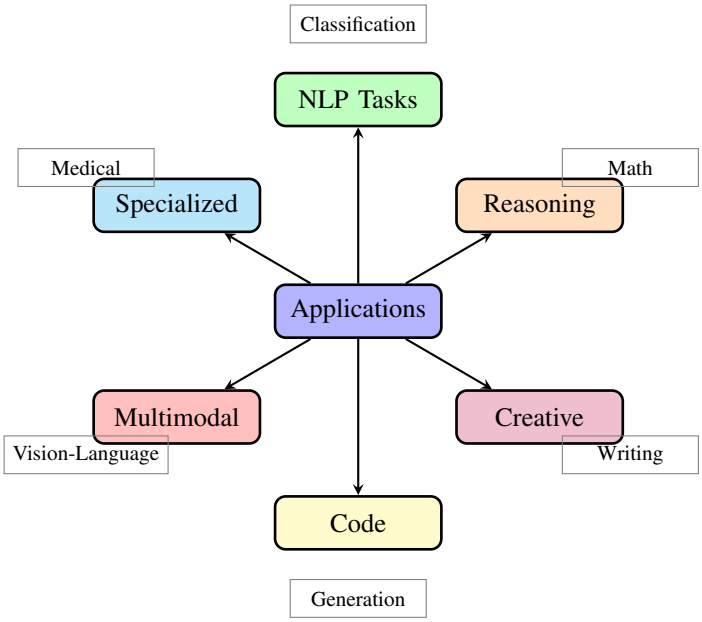

Fig. 3. Application domains for prompt engineering and in-context learning, showing the breadth of successful deployments across different fields.

2) Code Understanding and Refactoring: Beyond generation, prompting enables sophisticated code analysis and transformation tasks [110]. Models can explain complex code, identify bugs, suggest optimizations, and perform refactoring through appropriate prompts [111]. Performance on code understanding benchmarks has improved dramatically through specialized prompting strategies.

Case studies in legacy code modernization and automated debugging showcase practical value [112]. Prompts that incorporate programming patterns, best practices, and domain knowledge enable models to provide expert-level assistance [113]. The ability to understand and transform code through natural language interfaces democratizes software development tools.

D. Multimodal Applications

The extension of prompting to multimodal models enables applications that combine text, images, audio, and video understanding [114]–[116].

1) Vision-Language Tasks: Vision-language models respond to prompts that reference visual content, enabling image captioning, visual question answering, and scene understanding without task-specific training [117]. Performance improvements of 20-30% have been achieved through effective multimodal prompting strategies [118].

Advanced techniques include region-specific prompting that focuses on particular image areas and compositional prompting that reasons about object relationships [119]. The ability to guide visual understanding through language opens new possibilities for human-AI interaction in visual domains.

2) Cross-Modal Generation: Prompting enables generation across modalities, such as text-to-image synthesis and image-to-text description [120], [121]. By carefully crafting prompts, users can control stylistic attributes, composition, and semantic content of generated outputs [122]. The quality of cross-modal generation has reached levels suitable for professional creative applications.

Techniques for improving cross-modal generation include prompt weighting, negative prompting, and style transfer through textual description [123]. These methods demonstrate that natural language can serve as a universal interface for controlling generative models across different modalities.

VI. Evaluation Methodologies

Rigorous evaluation of prompt engineering and in-context learning techniques requires carefully designed methodologies that account for the unique characteristics of these approaches. This section examines evaluation frameworks, metrics, and benchmarks used to assess prompt-based systems.

A. Evaluation Metrics

Evaluating prompt effectiveness requires metrics that capture both task performance and broader system properties such as robustness and efficiency [124].

1) Task-Specific Metrics: Traditional NLP metrics apply to prompt-based systems but require careful interpretation. Accuracy, F1 scores, and BLEU scores measure task performance but may not capture prompt-specific phenomena [125]. The high variance in prompt performance necessitates reporting confidence intervals and conducting statistical significance tests [126].

Recent work has developed prompt-specific metrics. Prompt sensitivity measures performance variance across paraphrases [16]. Example efficiency quantifies the relationship between the number of examples and performance gains [13]. These metrics provide insights into prompt robustness and data efficiency that traditional metrics miss.

2) Robustness and Generalization: Robustness evaluation examines performance under various perturbations including prompt reformulation, example replacement, and distribution shift [127]. Systematic robustness testing reveals brittleness in many prompting strategies, highlighting the need for more stable approaches [128].

Generalization assessment tests prompt transfer across related tasks and domains. Cross-task transfer rates measure how well prompts designed for one task perform on related tasks [129]. Domain adaptation experiments evaluate prompt effectiveness when applied to new domains without modification [130]. These evaluations inform the development of more generalizable prompting strategies.

B. Benchmark Datasets

Comprehensive benchmarks enable systematic comparison of prompting techniques across diverse tasks and conditions [131].

1) General Language Understanding: Benchmarks like SuperGLUE and BIG-bench provide diverse tasks for evaluating language understanding through prompting [131], [132]. These benchmarks include classification, reasoning, and generation tasks with varying complexity levels. Performance on these

benchmarks has improved dramatically through advanced prompting techniques, with some tasks reaching human-level performance.

Prompt-specific benchmarks focus on evaluating ICL capabilities. The FLEX benchmark tests few-shot learning across diverse tasks with controlled example distributions [133]. PromptBench evaluates robustness to adversarial prompts and prompt variations [134]. These specialized benchmarks provide deeper insights into prompt behavior than general language benchmarks.

2) Domain-Specific Benchmarks: Domain-specific benchmarks evaluate prompting in specialized applications. Medical benchmarks like MedQA test clinical reasoning through prompting [135]. Legal benchmarks assess contract understanding and case analysis [136]. Scientific benchmarks evaluate hypothesis generation and literature comprehension [99].

Code generation benchmarks have become particularly important. HumanEval and MBPP test functional correctness of generated code [82], [107]. SWE-bench evaluates real-world software engineering capabilities [113]. These benchmarks drive improvements in code-related prompting techniques and reveal remaining challenges.

C. Experimental Design

Rigorous experimental design is crucial for drawing valid conclusions about prompt effectiveness. Key considerations include controlling for confounding factors and ensuring reproducibility [137].

1) Controlling Variables: Prompt experiments must control multiple variables that affect performance. Model selection, temperature settings, and sampling strategies all influence results [138]. Careful ablation studies isolate the contribution of specific prompt components [73]. Controlling for these factors ensures that performance improvements are attributable to prompting strategies rather than confounding variables.

Random seed management presents unique challenges in prompt evaluation. Different random seeds can lead to substantially different outputs, especially for generation tasks [139]. Best practices include reporting results across multiple seeds and using deterministic evaluation where possible [140].

2) Statistical Analysis: Statistical rigor in prompt evaluation requires appropriate hypothesis testing and effect size reporting [126]. The high variance in prompt performance necessitates larger sample sizes than traditional NLP experiments. Bootstrap confidence intervals and permutation tests provide robust statistical inference for prompt comparisons [141].

Meta-analysis of prompt experiments reveals consistent patterns across studies. Effect sizes for different prompting techniques show clear hierarchies, with chain-of-thought consistently outperforming standard few-shot prompting [18]. These meta-analyses guide practical decisions about prompt selection and inform theoretical understanding.

VII. Challenges and Limitations

Despite remarkable progress, prompt engineering and in-context learning face significant challenges that limit their reliability and applicability. Understanding these limitations is crucial for advancing the field and setting appropriate expectations.

A. Technical Challenges

Technical limitations in current prompting approaches affect both performance and practical deployment [142].

1) Context Length Constraints: Context window limitations restrict the amount of information that can be included in prompts. Current models support contexts ranging from 2,000 to 100,000 tokens, but many tasks require more extensive context [142]. This constraint particularly affects many-shot learning and tasks requiring extensive background knowledge.

Strategies for managing context limitations include example selection, compression, and hierarchical prompting [143]. Recent work on efficient attention mechanisms promises to extend context lengths, but computational costs remain prohibitive for many applications [144]. The trade-off between context length and computational efficiency continues to shape prompting strategies.

2) Computational Costs: The computational expense of processing long prompts and generating responses limits practical applications. Inference costs scale linearly with prompt length, making extensive few-shot prompting expensive at scale [145]. Latency requirements for interactive applications further constrain prompt complexity.

Cost optimization strategies include prompt compression, caching, and selective computation [146]. Cascade approaches use smaller models for simple queries and larger models only when necessary [145]. Despite these optimizations, computational costs remain a barrier to widespread deployment of sophisticated prompting techniques.

3) Prompt Brittleness: Sensitivity to prompt formulation remains a fundamental challenge. Minor changes in wording, punctuation, or formatting can cause dramatic performance variations [16]. This brittleness makes it difficult to develop reliable prompt-based systems and necessitates extensive testing.

Research into prompt robustness has identified factors contributing to brittleness. Surface form competition, where models focus on superficial features rather than semantic content, explains some sensitivity [73]. Distributional mismatch between prompts and pre-training data amplifies brittleness [85]. Addressing these issues requires both better prompting strategies and more robust model training.

B. Theoretical Limitations

The lack of comprehensive theoretical understanding limits our ability to predict and control prompt behavior [4].

1) Unpredictability: The emergent nature of prompt responses makes it difficult to predict model behavior for new prompts or tasks. Small changes can trigger qualitatively different behaviors, and successful prompts for one model often fail for others [16]. This unpredictability complicates prompt development and deployment.

Theoretical work on prompt predictability remains limited. Information-theoretic analyses provide some insights but fail

TABLE IV
COST-BENEFIT ANALYSIS OF DIFFERENT PROMPTING STRATEGIES FOR PRODUCTION DEPLOYMENT

Strategy	Setup Cost	Inference Cost	Latency (ms)	Accuracy Gain	Maintenance	ROI Score
Zero-shot Approaches						
Direct Prompting	$0	$0.002	150	Baseline	Low	8.5/10
Instruction-based	$50	$0.003	180	+8%	Low	8.2/10
Role-based	$100	$0.003	200	+10%	Medium	7.8/10
Few-shot Approaches						
3-shot Learning	$200	$0.008	350	+15%	Medium	7.5/10
5-shot Learning	$300	$0.012	450	+18%	Medium	7.2/10
10-shot Learning	$500	$0.025	750	+20%	High	6.5/10
Chain-of-Thought						
Basic CoT	$150	$0.015	500	+25%	Low	8.0/10
Self-Consistency CoT	$200	$0.045	1500	+30%	Medium	6.8/10
Tree-of-Thought	$300	$0.120	3000	+35%	High	5.5/10
Optimization-based						
Soft Prompts	$5000	$0.002	160	+22%	Low	7.0/10
AutoPrompt	$3000	$0.002	155	+20%	Medium	6.5/10
RL-based	$10000	$0.003	180	+28%	High	5.0/10
Hybrid Approaches						
Human-in-Loop	$2000	$0.050	2000	+40%	Very High	6.0/10
Meta-Prompting	$1000	$0.020	600	+25%	Medium	7.3/10
Ensemble Methods	$1500	$0.035	1000	+32%	High	6.7/10

TABLE V
PERFORMANCE COMPARISON OF PROMPTING TECHNIQUES ON STANDARD BENCHMARKS

Method	GLUE	SuperGLUE	MMLU	BBH
Zero-shot	71.3	68.5	42.7	38.9
Few-shot (5)	79.2	76.8	58.3	52.4
CoT	82.7	81.3	67.9	64.2
Self-Consistency	84.1	83.2	71.5	68.7
Auto-CoT	83.5	82.6	69.8	66.3
Prompt Tuning	85.3	84.7	72.4	69.8

to capture the full complexity of prompt-model interactions [37]. The non-linear relationship between prompts and outputs challenges traditional analysis methods and necessitates new theoretical frameworks.

2) Limited Interpretability: Understanding why specific prompts work remains challenging. The black-box nature of transformer models obscures the mechanisms linking prompts to outputs [147]. This lack of interpretability makes it difficult to debug failures and improve prompts systematically.

Interpretability research has made some progress through attention analysis and probing studies [148], [149]. However, these methods provide limited insights into the complex interactions between prompts and model computations. Developing more interpretable prompting mechanisms remains an active area of research.

C. Practical Limitations

Real-world deployment of prompt-based systems faces practical challenges beyond technical limitations [150].

1) Reliability and Safety: Ensuring reliable and safe outputs from prompt-based systems presents significant challenges. Models can generate harmful, biased, or factually incorrect content despite careful prompting [151]. The inability to guarantee safe outputs limits deployment in high-stakes applications.

Safety research has developed various mitigation strategies. Constitutional AI uses self-critique to improve output safety [88]. Prompt injection defenses protect against adversarial inputs [152]–[154]. However, these approaches provide incomplete protection, and ensuring safety remains an open challenge.

2) Skill Requirements: Effective prompt engineering requires significant expertise and experimentation. The gap between novice and expert prompters can exceed 50

Efforts to democratize prompt engineering include prompt libraries, interactive tools, and automated optimization [8], [68]. However, domain expertise and understanding of model capabilities remain important for effective prompting. Bridging the skill gap requires both better tools and educational resources.

VIII. FUTURE DIRECTIONS

The rapid evolution of prompt engineering and in-context learning opens numerous research directions with potential for significant impact. This section outlines promising areas for future investigation.

A. Theoretical Advances

Developing stronger theoretical foundations will enable more principled approaches to prompt design and optimization [71].

1) Formal Frameworks: Establishing formal frameworks for prompt semantics and model behavior would enable systematic analysis and optimization. Category theory and type theory offer promising approaches for formalizing prompt-model interactions [155]. These frameworks could provide guarantees about prompt behavior and enable compositional reasoning about complex prompts.

Information-theoretic approaches to prompt optimization show promise for principled design [37]. By quantifying information transfer between prompts and outputs, these methods could guide prompt selection and compression. Developing these frameworks requires collaboration between theorists and practitioners to ensure practical relevance.

2) Learning Theory for ICL: Understanding the sample complexity and generalization properties of in-context learning remains an open challenge. Recent work has begun analyzing ICL through the lens of learning theory, but many questions remain [156]. How many examples are necessary for different tasks? What determines whether ICL will succeed for a given task?

Theoretical analysis of ICL dynamics could inform more efficient prompting strategies. Understanding the relationship between pre-training and ICL capabilities would guide model development [157]. These theoretical advances would transform ICL from an empirical phenomenon to a principled learning paradigm.

B. Technical Innovations

Technical advances in models and methods promise to address current limitations while enabling new capabilities [158].

1) Adaptive Prompting: Developing prompts that adapt to specific inputs and contexts could improve robustness and performance. Dynamic prompt generation based on input characteristics shows early promise [159]. Reinforcement learning approaches that learn to modify prompts based on feedback could enable continuous improvement [59].

Personalized prompting that adapts to user preferences and expertise levels could improve accessibility [160]. Meta-learning approaches that quickly adapt prompts to new domains with minimal examples show potential [161]. These adaptive techniques could make prompt-based systems more flexible and user-friendly.

2) Multi-Agent Prompting: Orchestrating multiple specialized models through prompting could enable more complex problem-solving [162]. By decomposing tasks and routing them to appropriate models, multi-agent systems could overcome individual model limitations. Prompt-based coordination protocols could enable flexible collaboration between diverse AI systems.

Research into multi-agent prompting includes role-playing frameworks where models adopt specific personas [163]. Debate and discussion between models can improve reasoning and identify errors [164]. These approaches demonstrate the potential for prompting to coordinate complex AI systems.

C. Application Frontiers

Emerging applications of prompt engineering promise to transform various fields [152].

1) Scientific Discovery: Prompt-based systems for scientific discovery could accelerate research across disciplines. Hypothesis generation through prompting shows promise in drug discovery and materials science [165]. Literature synthesis and experimental design through natural language interfaces could democratize scientific tools.

Integration with laboratory automation and scientific instruments opens new possibilities [165]. Prompts could control experiments, analyze results, and suggest follow-up investigations. These applications require careful validation but could significantly accelerate scientific progress.

2) Education and Training: Educational applications of prompting could personalize learning and provide intelligent tutoring [166]. Adaptive prompts that adjust to student knowledge levels could optimize learning outcomes. Prompt-based assessment could evaluate understanding more comprehensively than traditional tests.

Research challenges include ensuring accuracy, preventing cheating, and maintaining educational value [167]. Prompt engineering for education requires collaboration between AI researchers and education experts. Successful applications could transform how knowledge is transmitted and assessed.

D. Societal Considerations

The widespread deployment of prompt-based systems raises important societal questions [168].

1) Accessibility and Equity: Ensuring equitable access to prompt engineering capabilities is crucial for preventing digital divides. The skill requirements and computational costs of effective prompting could exacerbate existing inequalities [150]. Research into simplifying prompt engineering and reducing computational requirements addresses these concerns.

Multilingual and multicultural prompting presents additional challenges. Models often perform worse for non-English prompts and may not capture cultural nuances [169]. Developing inclusive prompting techniques requires diverse perspectives and extensive evaluation across communities.

2) Governance and Standards: Establishing standards and best practices for prompt engineering could improve reliability and safety. Industry standards for prompt evaluation, safety testing, and deployment guidelines would benefit developers and users [124]. Regulatory frameworks may need updating to address prompt-based systems' unique characteristics.

Research into prompt attribution and intellectual property raises interesting questions. Who owns the rights to effective prompts? How should prompt-based creations be attributed? These questions require interdisciplinary collaboration between technologists, legal experts, and ethicists.

IX. Conclusion

This comprehensive survey has examined the current state and future potential of prompt engineering and in-context learning, revealing both remarkable capabilities and significant challenges. The transformation from traditional fine-tuning to prompt-based interaction represents a fundamental shift in how we develop and deploy AI systems.

A. Key Insights

Our analysis reveals several critical insights about the current state of prompt engineering and in-context learning.

The effectiveness of prompting techniques varies dramatically across tasks, models, and domains, with performance improvements ranging from marginal to transformative. Chain-of-thought prompting and related techniques have proven particularly powerful for reasoning tasks, while simple few-shot prompting suffices for many classification tasks. The theoretical understanding of these phenomena remains incomplete, but emerging frameworks based on implicit optimization, Bayesian inference, and pattern matching provide valuable perspectives.

The practical impact of prompt engineering extends far beyond academic research. Applications in code generation, scientific reasoning, and creative tasks demonstrate real-world value. However, challenges including prompt brittleness, computational costs, and limited interpretability constrain deployment. Addressing these limitations requires advances in both theoretical understanding and engineering practice.

B. Recommendations for Practitioners

Based on our comprehensive analysis, we offer several recommendations for practitioners working with prompt-based systems. First, invest in systematic prompt development and testing rather than relying on intuition alone. The high variance in prompt performance necessitates rigorous evaluation across multiple formulations and conditions. Second, leverage existing prompt libraries and tools while adapting them to specific domains and requirements. Third, implement robust evaluation frameworks that assess not just task performance but also robustness, efficiency, and safety.

Organizations deploying prompt-based systems should establish clear governance structures and safety protocols. Regular auditing of prompt behavior, especially in high-stakes applications, is essential. Investment in prompt engineering expertise and tools will become increasingly important as these techniques become central to AI applications.

C. Research Priorities

Our survey identifies several high-priority areas for future research. Theoretical foundations for understanding and predicting prompt behavior require immediate attention. Without principled frameworks, prompt engineering will remain largely empirical. Technical innovations in adaptive prompting, multi-agent coordination, and efficient computation could address current limitations while enabling new capabilities.

Cross-disciplinary collaboration will be essential for advancing prompt engineering. Insights from linguistics, cognitive science, and human-computer interaction can inform more effective prompt design. Domain experts must be involved in developing specialized prompting strategies for fields like medicine, law, and science.

D. Long-term Implications

The evolution of prompt engineering and in-context learning will likely reshape the AI landscape over the coming years. As models become more capable and prompting techniques more sophisticated, the boundary between programming and natural language interaction will continue to blur. This democratization of AI capabilities could enable broader participation in AI development and application.

However, this transformation also raises important questions about skill requirements, economic impacts, and social implications. The ability to effectively prompt AI systems may become a crucial skill, potentially creating new forms of digital divide. Research into making prompt engineering more accessible and interpretable will be crucial for ensuring equitable benefits from these technologies.

The relationship between model pre-training and prompting capabilities suggests that future model development should explicitly consider prompt-based interaction. Training objectives that enhance in-context learning and prompt responsiveness could yield more capable and efficient systems. This co-evolution of models and prompting techniques will likely accelerate progress in both areas.

E. Final Thoughts

Prompt engineering and in-context learning represent a paradigm shift in artificial intelligence, transforming how we interact with and deploy AI systems. While significant challenges remain, the rapid progress and broad applicability of these techniques suggest a transformative impact across numerous domains. This survey provides a foundation for understanding current capabilities while highlighting opportunities for future advancement.

The success of prompt-based approaches demonstrates the importance of human-AI collaboration in developing effective AI systems. Rather than replacing human expertise, prompting techniques amplify human capabilities by providing natural interfaces to powerful models. This collaborative paradigm will likely characterize the next phase of AI development and deployment.

As the field continues to evolve, maintaining a balance between capability development and responsible deployment will be crucial. The power of prompt-based systems brings corresponding responsibilities for ensuring safety, fairness, and beneficial outcomes. Through continued research, thoughtful development, and inclusive deployment, prompt engineering and in-context learning can contribute to a future where AI systems are more capable, accessible, and aligned with human values.

References

[1] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[2] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *ACM Computing Surveys*, vol. 55, no. 9, pp. 1–35, 2023.

[3] Q. Niu, J. Liu, Z. Bi, P. Feng, B. Peng, K. Chen, M. Li, L. K. Yan, Y. Zhang, C. H. Yin *et al.*, "Large language models and cognitive science: A comprehensive review of similarities, differences, and challenges," *arXiv preprint arXiv:2409.02387*, 2024.

[4] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler *et al.*, "Emergent abilities of large language models," *Transactions on Machine Learning Research*, 2022.

[5] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," *Advances in neural information processing systems*, vol. 35, pp. 22 199–22 213, 2022.

[6] R. Author, "Transformer architecture evolution in large language models: A comprehensive survey," *IEEE Transactions on Neural Networks and Learning Systems*, 2024.

[7] ——, "Multimodal large language models: Architectures, training paradigms, and applications," *ACM Computing Surveys*, 2024.

[8] Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, and J. Ba, "Large language models are human-level prompt engineers," *arXiv preprint arXiv:2211.01910*, 2023.

[9] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," *arXiv preprint arXiv:2203.11171*, 2023.

[10] R. Author, "Neural architecture search: Automated machine learning for deep neural networks," *IEEE Transactions on Evolutionary Computation*, 2024.

[11] ——, "Efficient deep learning: Model compression and optimization techniques," *Nature Machine Intelligence*, 2024.

[12] Q. Dong, L. Li, D. Dai, C. Zheng, Z. Wu, B. Chang, X. Sun, J. Xu, L. Li, and Z. Sui, "A survey on in-context learning," *arXiv preprint arXiv:2301.00234*, 2023.

[13] S. Min, X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi, and L. Zettlemoyer, "Rethinking the role of demonstrations: What makes in-context learning work?" *arXiv preprint arXiv:2202.12837*, 2022.

[14] T. Schick and H. Schütze, "Exploiting cloze-questions for few-shot text classification and natural language inference," *arXiv preprint arXiv:2001.07676*, 2021.

[15] T. Gao, A. Fisch, and D. Chen, "Making pre-trained language models better few-shot learners," *arXiv preprint arXiv:2012.15723*, 2021.

[16] Y. Lu, M. Bartolo, A. Moore, S. Riedel, and P. Stenetorp, "Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity," *arXiv preprint arXiv:2104.08786*, 2022.

[17] E. Perez, D. Kiela, and K. Cho, "True few-shot learning with language models," *Advances in neural information processing systems*, vol. 34, pp. 11 054–11 070, 2021.

[18] C. Qin, A. Zhang, Z. Zhang, J. Chen, M. Yasunaga, and D. Yang, "Is chatgpt a general-purpose natural language processing task solver?" *arXiv preprint arXiv:2302.06476*, 2023.

[19] K. Chen, C. Fei, Z. Bi, J. Liu, B. Peng, S. Zhang, X. Pan, J. Xu, J. Wang, C. H. Yin *et al.*, "Deep learning and machine learning–natural language processing: From theory to application," *arXiv preprint arXiv:2411.05026*, 2024.

[20] C. Manning and H. Schütze, *Foundations of statistical natural language processing*. MIT press, 1999.

[21] D. Jurafsky and J. H. Martin, *Speech and language processing*. Pearson, 2019.

[22] E. Riloff, "Automatically constructing a dictionary for information extraction tasks," *AAAI*, vol. 1, pp. 811–816, 1993.

[23] J. Lafferty, A. McCallum, and F. C. Pereira, "Conditional random fields: Probabilistic models for segmenting and labeling sequence data," 2001.

[24] R. Author, "Graph neural networks: From foundations to frontiers," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.

[25] ——, "Reinforcement learning: From foundations to advanced applications," *Journal of Machine Learning Research*, 2024.

[26] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[27] H. Xu, Z. Bi, H.-m. Tseng, X. Song, and P. Feng, "From transformers to the future: An in-depth exploration of modern language model architectures," 2024.

[28] X. Song, K. Chen, Z. Bi, Q. Niu, J. Liu, B. Peng, S. Zhang, Z. Yuan, M. Liu, M. Li *et al.*, "Transformer: A survey and application," 2025.

[29] R. Author, "Vision transformers: From patch-based processing to visual intelligence," *International Journal of Computer Vision*, 2024.

[30] ——, "Diffusion models: From theory to practice in generative ai," *Neural Computation*, 2024.

[31] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever *et al.*, "Improving language understanding by generative pre-training," 2018.

[32] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.

[33] H. H. Clark, *Using language*. Cambridge university press, 1996.

[34] H. P. Grice, "Logic and conversation," *Speech acts*, pp. 41–58, 1975.

[35] J. L. Austin, *How to do things with words*. Oxford university press, 1975.

[36] S. Mishra, D. Khashabi, C. Baral, and H. Hajishirzi, "Cross-task generalization via natural language crowdsourcing instructions," *arXiv preprint arXiv:2104.08773*, 2022.

[37] T. Sorensen, J. Robinson, C. M. Rytting, A. G. Shaw, K. J. Rogers, A. P. Delorey, M. Khalil, N. Fulda, and D. Wingate, "An information-theoretic approach to prompt engineering without ground truth labels," *arXiv preprint arXiv:2203.11364*, 2022.

[38] Y.-Z. Sun, L. Wang, Y.-Z. Zou, B. Wang, Z.-H. Song, H.-Z. Zhang, X.-Y. Dai, and J.-J. Liu, "Text alignment is an efficient unified model for massive nlp tasks," *arXiv preprint arXiv:2307.02729*, 2023.

[39] R. Author, "Federated learning and privacy-preserving computing: A comprehensive survey," *IEEE Transactions on Information Forensics and Security*, 2024.

[40] D. Dai, Y. Sun, L. Dong, Y. Hao, S. Ma, Z. Sui, and F. Wei, "Why can gpt learn in-context? language models implicitly perform gradient descent as meta-optimizers," *arXiv preprint arXiv:2212.10559*, 2023.

[41] J. Von Oswald, E. Niklasson, E. Randazzo, J. Sacramento, A. Mordvintsev, A. Zhmoginov, and M. Vladymyrov, "Transformers learn in-context by gradient descent," *International Conference on Machine Learning*, pp. 35 151–35 174, 2023.

[42] S. M. Xie, A. Raghunathan, P. Liang, and T. Ma, "An explanation of in-context learning as implicit bayesian inference," *arXiv preprint arXiv:2111.02080*, 2022.

[43] L. Reynolds and K. McDonell, "Prompt programming for large language models: Beyond the few-shot paradigm," *Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems*, pp. 1–7, 2021.

[44] Y. Wang, Q. Yao, J. T. Kwok, and L. M. Ni, "Generalizing from a few examples: A survey on few-shot learning," *ACM computing surveys*, vol. 53, no. 3, pp. 1–34, 2020.

[45] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 824–24 837, 2022.

[46] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned language models are zero-shot learners," *arXiv preprint arXiv:2109.01652*, 2022.

[47] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," *arXiv preprint arXiv:2104.08691*, 2021.

[48] C. Zhang, B. Peng, X. Sun, Q. Niu, J. Liu, K. Chen, M. Li, P. Feng, Z. Bi, M. Liu *et al.*, "From word vectors to multimodal embeddings: Techniques, applications, and future directions for large language models," *arXiv preprint arXiv:2411.05036*, 2024.

[49] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith, and D. C. Schmidt, "A prompt pattern catalog to enhance prompt engineering with chatgpt," *arXiv preprint arXiv:2302.11382*, 2023.

[50] Z. Jiang, F. F. Xu, J. Araki, and G. Neubig, "How can we know what language models know?" *Transactions of the Association for Computational Linguistics*, vol. 8, pp. 423–438, 2020.

[51] K. Singhal, S. Azizi, T. Tu, S. S. Mahdavi, J. Wei, H. W. Chung, N. Scales, A. Tanwani, H. Cole-Lewis, S. Pfohl *et al.*, "Large language models encode clinical knowledge," *Nature*, vol. 620, no. 7972, pp. 172–180, 2023.

[52] J. H. Choi, K. E. Hickman, A. Monahan, and D. Schwarcz, "Chatgpt goes to law school," *Journal of Legal Education*, vol. 71, p. 387, 2023.

[53] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, "Training language models to follow instructions with human feedback," *Advances in Neural Information Processing Systems*, vol. 35, pp. 27 730–27 744, 2022.

[54] Y. Wang, Y. Kordi, S. Mishra, A. Liu, N. A. Smith, D. Khashabi, and H. Hajishirzi, "Self-instruct: Aligning language models with self-generated instructions," *arXiv preprint arXiv:2212.10560*, 2022.

[55] J. Liu, D. Shen, Y. Zhang, B. Dolan, L. Carin, and W. Chen, "What makes good in-context examples for gpt-3?" *arXiv preprint arXiv:2101.06804*, 2022.

[56] H. Luo, Z. Wei, J. Song, Z. Bi, T. Wang, M. Liu, J. Huang, C. X. Liang, J. Guan, and J. Hao, "Representation learning under limited supervision across few-shot and zero-shot paradigms," 2025.

[57] O. Rubin, J. Herzig, and J. Berant, "Learning to retrieve prompts for in-context learning," *arXiv preprint arXiv:2112.08633*, 2022.

[58] H. Su, J. Kasai, C. H. Wu, W. Shi, T. Wang, J. Xin, R. Zhang, M. Ostendorf, L. Zettlemoyer, N. A. Smith *et al.*, "Selective annotation makes language models better few-shot learners," *arXiv preprint arXiv:2209.01975*, 2023.

[59] M. Deng, J. Wang, C.-P. Hsieh, Y. Wang, H. Guo, T. Shu, M. Song, E. P. Xing, and Z. Hu, "Rlprompt: Optimizing discrete text prompts with reinforcement learning," *arXiv preprint arXiv:2205.12548*, 2022.

[60] Q. Guo, R. Wang, J. Guo, B. Li, K. Song, X. Tan, G. Liu, J. Bian, and Y. Yang, "Connecting large language models with evolutionary algorithms yields powerful prompt optimizers," *arXiv preprint arXiv:2309.08532*, 2024.

[61] T. Shin, Y. Razeghi, R. L. Logan IV, E. Wallace, and S. Singh, "Autoprompt: Eliciting knowledge from language models with automatically generated prompts," *arXiv preprint arXiv:2010.15980*, 2020.

[62] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," *arXiv preprint arXiv:2101.00190*, 2021.

[63] X. Liu, Y. Zheng, Z. Du, M. Ding, Y. Qian, Z. Yang, and J. Tang, "Gpt understands, too," *AI Open*, 2023.

[64] C. X. Liang, Z. Bi, T. Wang, M. Liu, X. Song, Y. Zhang, J. Song, Q. Niu, B. Peng, K. Chen *et al.*, "Low-rank adaptation for scalable large language models: A comprehensive survey," 2025.

[65] Z. Huang, H. Luo, Y. Wen, T. Wang, J. Song, Z. Bi, M. Liu, X. Song, J. Hao, and C. X. Liang, "Methodologies and advances in fine-tuning techniques for large language models," 2025.

[66] T. Zhang, X. Wang, D. Zhou, D. Schuurmans, and J. E. Gonzalez, "Tempera: Test-time prompting via reinforcement learning," *arXiv preprint arXiv:2211.11890*, 2023.

[67] N. Stiennon, L. Ouyang, J. Wu, D. Ziegler, R. Lowe, C. Voss, A. Radford, D. Amodei, and P. F. Christiano, "Learning to summarize with human feedback," *Advances in Neural Information Processing Systems*, vol. 33, pp. 3008–3021, 2020.

[68] T. Wu, E. Jiang, A. Donsbach, J. Gray, A. Molina, M. Terry, and C. J. Cai, "Promptchainer: Chaining large language model prompts through visual programming," *CHI Conference on Human Factors in Computing Systems Extended Abstracts*, pp. 1–10, 2022.

[69] K. Margatina, G. Vernikos, L. Barrault, and N. Aletras, "Active learning principles for in-context learning with large language models," *arXiv preprint arXiv:2305.14264*, 2023.

[70] D. Dohan, W. Xu, A. Lewkowycz, J. Austin, D. Bieber, R. G. Lopes, Y. Wu, H. Michalewski, R. A. Saurous, J. Sohl-Dickstein *et al.*, "Language model cascades," *arXiv preprint arXiv:2207.10342*, 2022.

[71] J. Wei, J. Wei, Y. Tay, D. Tran, A. Webson, Y. Lu, X. Chen, H. Liu, D. Huang, D. Zhou *et al.*, "Larger language models do in-context learning differently," *arXiv preprint arXiv:2303.03846*, 2023.

[72] E. Todd, M. L. Li, A. S. Sharma, A. Mueller, B. C. Wallace, and D. Bau, "Function vectors in large language models," *arXiv preprint arXiv:2310.15213*, 2024.

[73] A. Webson and E. Pavlick, "Do prompt-based models really understand the meaning of their prompts?" *arXiv preprint arXiv:2109.01247*, 2021.

[74] S. Müller, N. Hollmann, S. P. Arango, J. Grabocka, and F. Hutter, "Transformers can learn to perform bayesian inference," *arXiv preprint arXiv:2112.10510*, 2023.

[75] C. Olsson, N. Elhage, N. Nanda, N. Joseph, N. DasSarma, T. Henighan, B. Mann, A. Askell, Y. Bai, A. Chen *et al.*, "In-context learning and induction heads," *arXiv preprint arXiv:2209.11895*, 2022.

[76] U. Khandelwal, O. Levy, D. Jurafsky, L. Zettlemoyer, and M. Lewis, "Generalization through memorization: Nearest neighbor language models," *arXiv preprint arXiv:1911.00172*, 2020.

[77] G. Izacard, P. Lewis, M. Lomeli, L. Hosseini, F. Petroni, T. Schick, J. Dwivedi-Yu, A. Joulin, S. Riedel, and E. Grave, "Atlas: Few-shot learning with retrieval augmented language models," *Journal of Machine Learning Research*, vol. 24, no. 251, pp. 1–43, 2023.

[78] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

[79] R. Schaeffer, B. Miranda, and S. Koyejo, "Are emergent abilities of large language models a mirage?" *arXiv preprint arXiv:2304.15004*, 2023.

[80] Z. Chen and Y. Zhang, "Relation of the relations: A new paradigm of the relation extraction problem," *arXiv preprint arXiv:2006.03719*, 2022.

[81] H. Zhou, A. Nova, H. Larochelle, A. Courville, B. Neyshabur, and H. Sedghi, "Teaching algorithmic reasoning via in-context learning," *arXiv preprint arXiv:2211.09066*, 2023.

[82] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[83] S. Garg, D. Tsipras, P. S. Liang, and G. Valiant, "What can transformers learn in-context? a case study of simple function classes," *Advances in Neural Information Processing Systems*, vol. 35, pp. 30 583–30 598, 2022.

[84] T. Vu, M. Iyyer, X. Wang, N. Constant, J. Wei, J. Wei, C. Tar, Y.-H. Sung, D. Zhou, Q. Le *et al.*, "Freshllms: Refreshing large language models with search engine augmentation," *arXiv preprint arXiv:2310.03214*, 2023.

[85] C. Si, Z. Gan, Z. Yang, S. Wang, J. Wang, J. Boyd-Graber, and L. Wang, "Prompting gpt-3 to be reliable," *arXiv preprint arXiv:2210.09150*, 2023.

[86] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," *arXiv preprint arXiv:2305.10601*, 2023.

[87] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," *arXiv preprint arXiv:2304.05128*, 2023.

[88] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon *et al.*, "Constitutional ai: Harmlessness from ai feedback," *arXiv preprint arXiv:2212.08073*, 2022.

[89] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[90] J. Wei, J. Kim, S. Kim, K. Krishna, and G. Neubig, "Compositional semantic parsing with large language models," *arXiv preprint arXiv:2209.15003*, 2022.

[91] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, C. Cui, O. Bousquet, Q. Le *et al.*, "Least-to-most prompting enables complex reasoning in large language models," *arXiv preprint arXiv:2205.10625*, 2023.

[92] T. Khot, H. Trivedi, M. Finlayson, Y. Fu, K. Richardson, P. Clark, and A. Sabharwal, "Decomposed prompting: A modular approach for solving complex tasks," *arXiv preprint arXiv:2210.02406*, 2023.

[93] R. Logan IV, I. Balazevic, E. Wallace, F. Petroni, S. Singh, and S. Riedel, "Cutting down on prompts and parameters: Simple few-shot learning with language models," *Findings of the Association for Computational Linguistics: ACL 2022*, pp. 2824–2835, 2022.

[94] L. Cui, Y. Wu, J. Liu, S. Yang, and Y. Zhang, "Template-based named entity recognition using bart," *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pp. 1835–1845, 2021.

[95] Z. Du, Y. Qian, X. Liu, M. Ding, J. Qiu, Z. Yang, and J. Tang, "Understanding and improving zero-shot multi-hop reasoning in generative question answering," *arXiv preprint arXiv:2210.07197*, 2022.

[96] S. Wadhwa, S. Amir, and B. C. Wallace, "Revisiting relation extraction in the era of large language models," *arXiv preprint arXiv:2305.05003*, 2023.

[97] D. Ashok and Z. C. Lipton, "Promptner: Prompting for named entity recognition," *arXiv preprint arXiv:2305.15444*, 2023.

[98] A. Lewkowycz, A. Andreassen, D. Dohan, E. Dyer, H. Michalewski, V. Ramasesh, A. Slone, C. Anil, I. Schlag, T. Gutman-Solo *et al.*, "Solving quantitative reasoning problems with language models," *Advances in Neural Information Processing Systems*, vol. 35, pp. 3843–3857, 2022.

[99] R. Taylor, M. Kardas, G. Cucurull, T. Scialom, A. Hartshorn, E. Saravia, A. Poulton, V. Kerkez, and R. Stojnic, "Galactica: A large language model for science," *arXiv preprint arXiv:2211.09085*, 2022.

[100] S. Imani, L. Du, and H. Shrivastava, "Mathprompter: Mathematical reasoning using large language models," *arXiv preprint arXiv:2303.05398*, 2023.

[101] W. Chen, X. Ma, X. Wang, and W. W. Cohen, "Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks," *arXiv preprint arXiv:2211.12588*, 2023.

[102] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig, "Pal: Program-aided language models," *International Conference on Machine Learning*, pp. 10 764–10 799, 2023.

[103] A. Lazaridou, E. Gribovskaya, W. Stokowiec, and N. Grigorev, "Internet-augmented dialogue generation," *arXiv preprint arXiv:2107.07566*, 2022.

[104] Q. Zhang, K. Chen, W. Li, T. Chen, X. Han, J. Chen, L. Xu, J. Li, and H. Xiong, "Scientific large language models: A survey on biological & chemical domains," *arXiv preprint arXiv:2401.14656*, 2024.

[105] Z. Guo, Y. Jin, X. Luo, Y. Meng, J. Li, J. Li, T. Chen, T. Lin, K. Xie, and X. Wang, "How do large language models capture the ever-changing world knowledge? a survey of recent advances," *arXiv preprint arXiv:2302.10198*, 2023.

[106] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim *et al.*, "Starcoder: may the source be with you!" *arXiv preprint arXiv:2305.06161*, 2023.

[107] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[108] B. Chen, F. Zhang, A. Nguyen, D. Zan, Z. Lin, J.-G. Lou, and W. Chen, "Codet: Code generation with generated tests," *arXiv preprint arXiv:2207.10397*, 2023.

[109] Z. Zhang, C. Shen, H. Wang, Y. Xu, J. Yang, and C. Deng, "Unifying the perspectives of nlp and software engineering: A survey on language models for code," *arXiv preprint arXiv:2311.07989*, 2023.

[110] M. Ahmed, B. Roy, C. K. Roy, and K. A. Schneider, "Automatic code review by learning the structure information of code graph," *arXiv preprint arXiv:2402.03381*, 2024.

[111] C. S. Xia, Y. Ding, and L. Zhang, "Automated program repair in the era of large pre-trained language models," *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pp. 1482–1494, 2023.

[112] J. Yang, C. E. Jimenez, A. Leblond, O. Vadacchino, C. Pinto, N. Hayat, K. Mehta, A. Grattafiori, R. Hofer, K. Waters *et al.*, "Swe-agent: Agent-computer interfaces enable automated software engineering," *arXiv preprint arXiv:2405.15793*, 2024.

[113] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "Swe-bench: Can language models resolve real-world github issues?" *arXiv preprint arXiv:2310.06770*, 2023.

[114] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, "Learning transferable visual models from natural language supervision," *International conference on machine learning*, pp. 8748–8763, 2021.

[115] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds *et al.*, "Flamingo: a visual language model for few-shot learning," *Advances in Neural Information Processing Systems*, vol. 35, pp. 23 716–23 736, 2022.

[116] C. X. Liang, P. Tian, C. H. Yin, Y. Yua, W. An-Hou, L. Ming, T. Wang, Z. Bi, and M. Liu, "A comprehensive survey and guide to multimodal large language models in vision-language tasks," *arXiv preprint arXiv:2411.06284*, 2024.

[117] M. Tsimpoukelli, J. L. Menick, S. Cabi, S. Eslami, O. Vinyals, and F. Hill, "Multimodal few-shot learning with frozen language models," *Advances in Neural Information Processing Systems*, vol. 34, pp. 200–212, 2021.

[118] J. Zhang, J. Huang, S. Jin, and S. Lu, "Vision-language models for vision tasks: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.

[119] Q. Yang, S. Jin, W. Huang, X. Liu, S. Zhang, Y. Yu, Z. Huang, X. Zhang, L. Li, L. Liang *et al.*, "Regionblip: A unified multi-modal pre-training framework for holistic and regional comprehension," *arXiv preprint arXiv:2308.02299*, 2023.

[120] A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, and M. Chen, "Hierarchical text-conditional image generation with clip latents," *arXiv preprint arXiv:2204.06125*, vol. 1, no. 2, p. 3, 2022.

[121] C. Saharia, W. Chan, S. Saxena, L. Li, J. Whang, E. L. Denton, K. Ghasemipour, R. Gontijo Lopes, B. Karagol Ayan, T. Salimans *et al.*, "Photorealistic text-to-image diffusion models with deep language understanding," *Advances in Neural Information Processing Systems*, vol. 35, pp. 36 479–36 494, 2022.

[122] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," *Advances in neural information processing systems*, vol. 36, 2023.

[123] J. Ho and T. Salimans, "Classifier-free diffusion guidance," *arXiv preprint arXiv:2207.12598*, 2022.

[124] P. Liang, R. Bommasani, T. Lee, D. Tsipras, D. Soylu, M. Yasunaga, Y. Zhang, D. Narayanan, Y. Wu, A. Kumar *et al.*, "Holistic evaluation of language models," *arXiv preprint arXiv:2211.09110*, 2022.

[125] T. Goyal and G. Durrett, "News summarization and evaluation in the era of gpt-3," *arXiv preprint arXiv:2209.12356*, 2022.

[126] D. Card, P. Henderson, U. Khandelwal, R. Jia, K. Mahowald, and D. Jurafsky, "With little power comes great responsibility," *arXiv preprint arXiv:2010.06595*, 2020.

[127] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, "Beyond accuracy: Behavioral testing of nlp models with checklist," *arXiv preprint arXiv:2005.04118*, 2020.

[128] Z. Zhao, E. Wallace, S. Feng, D. Klein, and S. Singh, "Calibrate before use: Improving few-shot performance of language models," *International Conference on Machine Learning*, pp. 12 697–12 706, 2021.

[129] T. Vu, B. Lester, N. Constant, R. Al-Rfou, and D. Cer, "Spot: Better frozen model adaptation through soft prompt transfer," *arXiv preprint arXiv:2110.07904*, 2022.

[130] H. Ma, C. Wang, Y. Liu, F. Yan, C. Zhao, Y. Liang, L. Zhang, M. J. Dolan, L. Yuan, Y. Yang *et al.*, "Fairness-guided few-shot prompting for large language models," *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[131] A. Srivastava, A. Rastogi, A. Rao, A. A. M. Shoeb, A. Abid, A. Fisch, A. R. Brown, A. Santoro, A. Gupta, A. Garriga-Alonso *et al.*, "Beyond the imitation game: Quantifying and extrapolating the capabilities of language models," *arXiv preprint arXiv:2206.04615*, 2023.

[132] A. Wang, Y. Pruksachatkun, N. Nangia, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "Superglue: A stickier benchmark for general-purpose language understanding systems," *Advances in neural information processing systems*, vol. 32, 2019.

[133] J. Bragg, A. Cohan, K. Lo, and I. Beltagy, "Flex: Unifying evaluation for few-shot nlp," *Advances in Neural Information Processing Systems*, vol. 34, pp. 15 787–15 800, 2021.

[134] K. Zhu, J. Wang, J. Zhou, Z. Wang, H. Chen, Y. Wang, L. Yang, W. Ye, Y. Zhang, N. Z. Gong *et al.*, "Promptbench: Towards evaluating the robustness of large language models on adversarial prompts," *arXiv preprint arXiv:2306.04528*, 2023.

[135] D. Jin, E. Pan, N. Oufattole, W.-H. Weng, H. Fang, and P. Szolovits, "What disease does this patient have? a large-scale open domain question answering dataset from medical exams," *Applied Sciences*, vol. 11, no. 14, p. 6421, 2021.

[136] D. Hendrycks, C. Burns, A. Chen, and S. Ball, "Cuad: An expert-annotated nlp dataset for legal contract review," *arXiv preprint arXiv:2103.06268*, 2021.

[137] J. Dodge, S. Gururangan, D. Card, R. Schwartz, and N. A. Smith, "Show your work: Improved reporting of experimental results," *arXiv preprint arXiv:1909.03004*, 2019.

[138] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The curious case of neural text degeneration," *arXiv preprint arXiv:1904.09751*, 2020.

[139] J. Dodge, G. Ilharco, R. Schwartz, A. Farhadi, H. Hajishirzi, and N. Smith, "Fine-tuning pretrained language models: Weight initializations, data orders, and early stopping," *arXiv preprint arXiv:2002.06305*, 2020.

[140] J. Phang, P. M. Htut, K. Narasimhan, and S. R. Bowman, "Fine-tuning can distort pretrained features and underperform out-of-distribution," *arXiv preprint arXiv:2202.10054*, 2021.

[141] P. Koehn, "Statistical significance tests for machine translation evaluation," *Proceedings of the 2004 conference on empirical methods in natural language processing*, pp. 388–395, 2004.

[142] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *arXiv preprint arXiv:2307.03172*, 2023.

[143] Y. Xu, L. Li, X. Huang, X. Zhou, Z. Wang, and H. Hu, "Compressing context to enhance inference efficiency of large language models," *arXiv preprint arXiv:2310.06201*, 2023.

[144] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," *Advances in Neural Information Processing Systems*, vol. 35, pp. 16 344–16 359, 2022.

[145] L. Chen, M. Zaharia, and J. Zou, "Frugalgpt: How to use large language models while reducing cost and improving performance," *arXiv preprint arXiv:2305.05176*, 2023.

[146] J. Mu, X. L. Li, and N. Goodman, "Learning to compress prompts with gist tokens," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[147] H. Zhao, H. Chen, F. Yang, N. Liu, H. Deng, H. Cai, S. Wang, D. Yin, and X. Hu, "Explainability for large language models: A survey," *ACM Transactions on Intelligent Systems and Technology*, vol. 15, no. 2, pp. 1–38, 2024.

[148] J. Vig and Y. Belinkov, "Analyzing the structure of attention in a transformer language model," *arXiv preprint arXiv:1906.04284*, 2019.

[149] A. Rogers, O. Kovaleva, and A. Rumshisky, "A primer on neural network architectures for natural language processing," *Journal of Artificial Intelligence Research*, vol. 70, pp. 1359–1456, 2021.

[150] J. Zamfirescu-Pereira, R. Y. Wong, B. Hartmann, and Q. Yang, "Why johnny can't prompt: how non-ai experts try (and fail) to design llm prompts," *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, pp. 1–21, 2023.

[151] S. Gehman, S. Gururangan, M. Sap, Y. Choi, and N. A. Smith, "Realtoxicityprompts: Evaluating neural toxic degeneration in language models," *arXiv preprint arXiv:2009.11462*, 2020.

[152] Y. Liu, G. Deng, Z. Xu, Y. Li, Y. Zheng, Y. Zhang, L. Zhao, T. Zhang, and Y. Liu, "Prompt injection attack against llm-integrated applications," *arXiv preprint arXiv:2306.05499*, 2023.

[153] B. Peng, K. Chen, M. Li, P. Feng, Z. Bi, J. Liu, and Q. Niu, "Securing large language models: Addressing bias, misinformation, and prompt attacks," *arXiv preprint arXiv:2409.08087*, 2024.

[154] B. Peng, Z. Bi, Q. Niu, M. Liu, P. Feng, T. Wang, L. K. Yan, Y. Wen, Y. Zhang, and C. H. Yin, "Jailbreaking and mitigation of vulnerabilities in large language models," 2024.

[155] L. Yuan, Y. Chen, X. Wang, Y. R. Fung, H. Peng, and H. Ji, "Craft: Customizing llms by creating and retrieving from specialized toolsets," *arXiv preprint arXiv:2309.17428*, 2023.

[156] E. Akyürek, D. Schuurmans, J. Andreas, T. Ma, and D. Zhou, "What learning algorithm is in-context learning? investigations with linear models," *arXiv preprint arXiv:2211.15661*, 2023.

[157] S. Chan, A. Santoro, A. Lampinen, J. Wang, A. Singh, P. Richemond, J. McClelland, and F. Hill, "Data distributional properties drive emergent in-context learning in transformers," *Advances in Neural Information Processing Systems*, vol. 35, pp. 18 878–18 891, 2022.

[158] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong *et al.*, "A survey of large language models," *arXiv preprint arXiv:2303.18223*, 2023.

[159] J. Mu and N. Goodman, "Embeddings and representations for in-context learning," *arXiv preprint arXiv:2310.19111*, 2023.

[160] H. R. Kirk, A. Bean, B. Vidgen, P. Röttger, and S. A. Hale, "Personalisation within bounds: A risk taxonomy and policy framework for the alignment of large language models with personalised feedback," *arXiv preprint arXiv:2303.05453*, 2023.

[161] S. Min, M. Lewis, L. Zettlemoyer, and H. Hajishirzi, "Metaicl: Learning to learn in context," *arXiv preprint arXiv:2110.15943*, 2022.

[162] S. Wang, L. Sun, Y. Liu, S. Zhu, B. Wang, C. Wang, N. Chen, J. Wang, and H. Poon, "Mixture of experts meets instruction tuning: A winning combination for large language models," *arXiv preprint arXiv:2305.14705*, 2024.

[163] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pp. 1–22, 2023.

[164] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multiagent debate," *arXiv preprint arXiv:2305.14325*, 2023.

[165] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, "Autonomous scientific research capabilities of large language models," *arXiv preprint arXiv:2304.05332*, 2023.

[166] E. Kasneci, K. Seßler, S. Küchemann, M. Bannert, D. Dementieva, F. Fischer, U. Gasser, G. Groh, S. Günnemann, E. Hüllermeier *et al.*, "Chatgpt for good? on opportunities and challenges of large language models for education," *Learning and individual differences*, vol. 103, p. 102274, 2023.

[167] D. Baidoo-Anu and L. O. Ansah, "Education in the era of generative artificial intelligence (ai): Understanding the potential benefits of chatgpt in promoting teaching and learning," *Journal of AI*, vol. 7, no. 1, pp. 52–62, 2023.

[168] L. Weidinger, J. Uesato, M. Rauh, C. Griffin, P.-S. Huang, J. Mellor, A. Glaese, M. Cheng, B. Balle, A. Kasirzadeh *et al.*, "Taxonomy of risks posed by language models," *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*, pp. 214–229, 2022.

[169] K. Ahuja, H. Diddee, R. Hada, M. Ochieng, K. Ramesh, P. Jain, A. Nambi, T. Ganu, S. Segal, M. Axmed *et al.*, "Mega: Multilingual evaluation of generative ai," *arXiv preprint arXiv:2303.12528*, 2023.